

Practical Week-4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
lenovo@Manya:~$ mkdir practical4
mkdir: practical4: File exists
lenovo@Manya:~$ cd practical4
lenovo@Manya:~/practical4$ nano task1.c
lenovo@Manya:~/practical4$ gcc task1.c -o task1
lenovo@Manya:~/practical4$ ./task1
Parent Process Started
Parent PID: 647
Child 1 started. PID = 648 Parent PID = 647
Child 2 started. PID = 649 Parent PID = 647
All 3 child processes have been created.

Using wait():
Child 3 started. PID = 650 Parent PID = 647
Child 1 completed. PID = 648
wait() detected child PID 648 finished with exit status 10

Using waitpid():
waitpid() detected child PID 648 finished with exit status 10
Child 2 completed. PID = 649
waitpid() detected child PID 649 finished with exit status 11
Child 3 completed. PID = 650
waitpid() detected child PID 650 finished with exit status 12
Parent process completed.
lenovo@Manya:~/practical4$ |
```

```
lenovo@Manya
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed. PID = %d\n",
                i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("\nUsing wait():\n");

    pid_t finished = wait(&status);

    printf("wait() detected child PID %d finished with exit status %d\n",
        finished, WEXITSTATUS(status));

    printf("\nUsing waitpid():\n");
```

```
for (int i = 0; i < 3; i++)
{
    waitpid(pid[i], &status, 0);

    printf("waitpid() detected child PID %d finished with exit status %d\n",
        pid[i], WEXITSTATUS(status));
}

printf("Parent process completed.\n");

return 0;
}
```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
lenovo@Manya:~/practical4$ nano task2.c
lenovo@Manya:~/practical4$ gcc task2.c -o task2
lenovo@Manya:~/practical4$ ./task2
Parent process: PID = 942
Child PID = 943
Parent sleeping...
Child process: PID = 943
Child process terminating...
Parent terminating...
lenovo@Manya:~/practical4$ ps -el
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	80	0	-	6027	-	?	00:00:00	system
5	S	0	2	1	0	80	0	-	795	-	hvc0	00:00:00	init-s
0	S	0	7	2	0	80	0	-	795	-	hvc0	00:00:00	init
4	S	0	47	1	0	79	-1	-	12607	-	?	00:00:00	system
4	S	989	82	1	0	80	0	-	5604	-	?	00:00:00	system
4	S	0	85	1	0	80	0	-	8862	-	?	00:00:01	system
1	S	0	152	1	0	80	0	-	39837	-	?	00:00:00	snapfu
1	S	0	155	1	0	80	0	-	113536	-	?	00:00:01	snapfu
1	S	0	158	1	0	80	0	-	39837	-	?	00:00:00	snapfu
4	S	0	187	1	0	80	0	-	722	-	?	00:00:00	chrony
4	S	0	188	1	0	80	0	-	1118	-	?	00:00:00	cron
4	S	995	189	1	0	80	0	-	2207	-	?	00:00:00	dbus-d
4	S	0	193	1	0	80	0	-	11024	-	?	00:00:00	network
4	S	0	196	1	0	80	0	-	555187	-	?	00:00:00	snappd
4	S	0	197	1	0	80	0	-	4610	-	?	00:00:00	system
4	S	0	201	1	0	80	0	-	448075	-	?	00:00:00	wsl-pr
4	S	100	216	1	0	80	0	-	55137	-	?	00:00:00	rsyslo
4	S	0	230	1	0	80	0	-	1373	-	tty1	00:00:00	agetty
4	S	988	249	187	0	80	0	-	5936	-	?	00:00:00	chrony
4	S	0	253	1	0	80	0	-	30865	-	?	00:00:00	unatte
1	S	988	265	249	0	80	0	-	3024	-	?	00:00:00	chrony
4	S	0	390	2	0	80	0	-	2162	-	?	00:00:00	login
4	S	1000	432	1	0	80	0	-	5589	-	?	00:00:00	system
5	S	1000	436	432	0	80	0	-	5729	-	?	00:00:00	(sd-pa
4	S	1000	460	390	0	80	0	-	1534	poll_s	pts/1	00:00:00	bash
5	S	0	835	2	0	80	0	-	797	-	?	00:00:00	Sessio
5	S	0	836	835	0	80	0	-	801	-	?	00:00:00	Relay(
4	S	1000	838	836	0	80	0	-	1567	do_wai	pts/0	00:00:00	bash
0	R	1000	971	838	0	80	0	-	1798	-	pts/0	00:00:00	ps

```
lenovo@Manya:~/practical4$
```



lenovo@Manya



GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");
        sleep(30);
        printf("Parent terminating...\n");
    }

    return 0;
}
```

```

lenovo@Manya:~$ cd practical4
lenovo@Manya:~/practical4$ nano task2.c
lenovo@Manya:~/practical4$ gcc task2.c -o task2
lenovo@Manya:~/practical4$ ./task2
Parent process: PID = 942
Child PID = 943
Parent sleeping...
Child process: PID = 943
Child process terminating...
Parent terminating...
lenovo@Manya:~/practical4$ ps -el

```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	80	0	-	6027	-	?	00:00:00	system
5	S	0	2	1	0	80	0	-	795	-	hvc0	00:00:00	init-s
0	S	0	7	2	0	80	0	-	795	-	hvc0	00:00:00	init
4	S	0	47	1	0	79	-1	-	12607	-	?	00:00:00	system
4	S	989	82	1	0	80	0	-	5604	-	?	00:00:00	system
4	S	0	85	1	0	80	0	-	8862	-	?	00:00:01	system
1	S	0	152	1	0	80	0	-	39837	-	?	00:00:00	snapfu
1	S	0	155	1	0	80	0	-	113536	-	?	00:00:01	snapfu
1	S	0	158	1	0	80	0	-	39837	-	?	00:00:00	snapfu
4	S	0	187	1	0	80	0	-	722	-	?	00:00:00	chrony
4	S	0	188	1	0	80	0	-	1118	-	?	00:00:00	cron
4	S	995	189	1	0	80	0	-	2207	-	?	00:00:00	dbus-d
4	S	0	193	1	0	80	0	-	11024	-	?	00:00:00	networ
4	S	0	196	1	0	80	0	-	555187	-	?	00:00:00	snappd
4	S	0	197	1	0	80	0	-	4610	-	?	00:00:00	system
4	S	0	201	1	0	80	0	-	448075	-	?	00:00:00	wsl-pr
4	S	100	216	1	0	80	0	-	55137	-	?	00:00:00	rsyslo
4	S	0	230	1	0	80	0	-	1373	-	tty1	00:00:00	agetty
4	S	988	249	187	0	80	0	-	5936	-	?	00:00:00	chrony
4	S	0	253	1	0	80	0	-	30865	-	?	00:00:00	unatte
1	S	988	265	249	0	80	0	-	3024	-	?	00:00:00	chrony
4	S	0	390	2	0	80	0	-	2162	-	?	00:00:00	login
4	S	1000	432	1	0	80	0	-	5589	-	?	00:00:00	system
5	S	1000	436	432	0	80	0	-	5729	-	?	00:00:00	(sd-pa
4	S	1000	460	390	0	80	0	-	1534	poll_s	pts/1	00:00:00	bash
5	S	0	835	2	0	80	0	-	797	-	?	00:00:00	Sessio
5	S	0	836	835	0	80	0	-	801	-	?	00:00:00	Relay(
4	S	1000	838	836	0	80	0	-	1567	do_wai	pts/0	00:00:00	bash
0	R	1000	971	838	0	80	0	-	1798	-	pts/0	00:00:00	ps

```

lenovo@Manya:~/practical4$ nano task2.c
lenovo@Manya:~/practical4$ nano task2.c
lenovo@Manya:~/practical4$ nano task2.c
lenovo@Manya:~/practical4$ gcc task2.c -o task2
lenovo@Manya:~/practical4$ ./task2
Child process terminating...
Zombie process removed.
lenovo@Manya:~/practical4$ |

```

```
lenovo@Manya × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process terminating...\n");

        exit(0);
    }
    else
    {
        wait(NULL);

        printf("Zombie process removed.\n");
    }

    return 0;
}
```